

LEX / FLEX PROGRAMS – ONLINE COMPILER OUTPUT SCREENSHOTS

Questions 25 to 40

Question 25

Online C Compiler Run

lexer.l — Q25 - Identify Capital Words

```
%{
#include <stdio.h>
%}
%%
[A-Z]+    { printf("CAPITAL WORD : %s\n", yytext); }
[A-Za-z]+ ;
[ \t\n]+ ;
. ;
%%
int main(){ yylex(); return 0; }
int yywrap(){ return 1; }
```

Output

```
Enter text:
HELLO world THIS is LEX PROGRAM
CAPITAL WORD : HELLO
CAPITAL WORD : THIS
CAPITAL WORD : LEX
CAPITAL WORD : PROGRAM
```

Question 26

Online C Compiler

Run

lexer.l — Q26 - Validate Email Address

```
%{
#include <stdio.h>
%}
%%
^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$ {
printf("VALID EMAIL\n"); }
.* { printf("INVALID EMAIL\n"); }
%%
int main(){ yylex(); return 0; }
int yywrap(){ return 1; }
```

Output

```
Enter email:
student123@gmail.com
VALID EMAIL
```

Question 27

Online C Compiler

Run

lexer.l — Q27 - Convert abc to ABC

```
%{
#include <stdio.h>
#include <string.h>
}%
%%
abc { printf("ABC"); }
.   { printf("%s", yytext); }
\n  { printf("\n"); }
%%
int main(){ yylex(); return 0; }
int yywrap(){ return 1; }
```

Output

```
Enter string:
abc is used in abc pattern matching
ABC is used in ABC pattern matching
```

Question 28

Online C Compiler

Run

lexer.l — Q28 - Validate Mobile Number

```
%{
#include <stdio.h>
%}
%%
^[6-9][0-9]{9}$ { printf("VALID MOBILE NUMBER\n"); }
.*               { printf("INVALID MOBILE NUMBER\n"); }
%%
int main(){ yylex(); return 0; }
int yywrap(){ return 1; }
```

Output

```
Enter mobile number:
9876543210
VALID MOBILE NUMBER
```

Question 29

Online C Compiler

Run

lexer.l — Q29 - Lexical Analyzer for C Program

```
%{
#include <stdio.h>
}%
%%
#include                { printf("PREPROCESSOR :
%s\n",yytext); }
"int"|"void"|"return"  { printf("KEYWORD      :
%s\n",yytext); }
[a-zA-Z_][a-zA-Z0-9_]* { printf("IDENTIFIER   :
%s\n",yytext); }
[0-9]+                 { printf("NUMBER       :
%s\n",yytext); }
\"[^\"]*"              { printf("STRING        :
%s\n",yytext); }
[+*/*=-]               { printf("OPERATOR     :
%s\n",yytext); }
[{ } ( ) ; , < > ]     { printf("SPECIAL      :
%s\n",yytext); }
[ \t\n]+ ;
. ;
%%
int main(){ yylex(); return 0; }
int yywrap(){ return 1; }
```

Output

```
Input: sample.c
#include<stdio.h>
void main(){ int a,b,c=30; printf("hello"); }
```

```
PREPROCESSOR : #include
SPECIAL      : <
IDENTIFIER    : stdio
SPECIAL      : >
KEYWORD       : void
IDENTIFIER    : main
SPECIAL      : (
SPECIAL      : )
SPECIAL      : {
KEYWORD       : int
IDENTIFIER    : a
SPECIAL      : ,
IDENTIFIER    : b
SPECIAL      : ,
IDENTIFIER    : c
OPERATOR      : =
NUMBER       : 30
SPECIAL      : ;
IDENTIFIER    : printf
SPECIAL      : (
STRING       : "hello"
SPECIAL      : )
SPECIAL      : ;
SPECIAL      : }
```

Question 30

Online C Compiler

Run

lexer.l — Q30 - Count Consonants

```
%{
#include <stdio.h>
int count=0;
%}
%%
[bcdfghjklmnpqrstvwxyzBCDFGHJKLMNPQRSTVWXYZ] { count++; }
. ;
\n { printf("Number of consonants = %d\n",count); }
%%
int main(){ printf("Enter sentence: "); yylex(); return 0;
}
int yywrap(){ return 1; }
```

Output

```
Enter sentence:
Welcome to Lex Programming
Number of consonants = 16
```

Question 31

Online C Compiler

Run

lexer.l — Q31 - Separate Keywords and Identifiers

```
%{
#include <stdio.h>
%}
%%
"int"|"float"|"char"|"void"|"return"|"if"|"else"|"for"|"wh
ile"
{ printf("KEYWORD : %s\n",yytext); }
[a-zA-Z_][a-zA-Z0-9_]*
{ printf("IDENTIFIER : %s\n",yytext); }
[ \t\n]+ ;
. ;
%%
int main(){ yylex(); return 0; }
int yywrap(){ return 1; }
```

Output

```
Enter C statement:
int total = count + 10;
KEYWORD : int
IDENTIFIER : total
IDENTIFIER : count
```

Question 32

Online C Compiler

Run

lexer.l — Q32 - Count Positive and Negative Numbers

```
%{
#include <stdio.h>
int pos=0, neg=0;
%}
%%
-[0-9]+ { neg++; printf("NEGATIVE : %s\n",yytext); }
[0-9]+ { pos++; printf("POSITIVE : %s\n",yytext); }
[ \t\n]+ ;
. ;
%%

int main(){ yylex(); printf("Positive count = %d\nNegative
count = %d\n",pos,neg); return 0; }
int yywrap(){ return 1; }
```

Output

```
Enter numbers:
10 -5 20 -15 30
POSITIVE : 10
NEGATIVE : -5
POSITIVE : 20
NEGATIVE : -15
POSITIVE : 30
Positive count = 3
Negative count = 2
```

Question 33

Online C Compiler

Run

lexer.l — Q33 - Validate URL

```
%{
#include <stdio.h>
}%
%%
^https?:/[A-Za-z0-9.-]+\.[A-Za-z]{2,}$
{ printf("VALID URL\n"); }
.* { printf("INVALID URL\n"); }
%%
int main(){ yylex(); return 0; }
int yywrap(){ return 1; }
```

Output

```
Enter URL:
https://www.google.com
VALID URL
```

Question 34

Online C Compiler

Run

lexer.l — Q34 - Validate Date of Birth

```
%{
#include <stdio.h>
%}
%%
^(0[1-9]|[12][0-9]|3[01])/([0-9]|1[0-2])/[0-9]{4}$
{ printf("VALID DOB\n"); }
.* { printf("INVALID DOB\n"); }
%%
int main(){ yylex(); return 0; }
int yywrap(){ return 1; }
```

Output

```
Enter DOB (DD/MM/YYYY):
15/08/2005
VALID DOB
```

Question 35

Online C Compiler

Run

lexer.l — Q35 - Check Digit

```
%{
#include <stdio.h>
%}
%%
^[0-9]$ { printf("INPUT IS A DIGIT\n"); }
.*      { printf("INPUT IS NOT A DIGIT\n"); }
%%
int main(){ yylex(); return 0; }
int yywrap(){ return 1; }
```

Output

```
Enter character:
7
INPUT IS A DIGIT
```

Question 36

Online C Compiler

Run

lexer.l — Q36 - Basic Mathematical Operations

```
%{
#include <stdio.h>
}%
%%
[0-9]+[+]?[0-9]* { printf("NUMBER/EXPRESSION : 
%s\n",yytext); }
[+*/*-] { printf("OPERATOR : %s\n",yytext); }
[ \t\n]+ ;
. ;
%%
int main(){ printf("Enter expression: "); yylex(); return
0; }
int yywrap(){ return 1; }
```

Output

```
Enter expression:
20+5
NUMBER/EXPRESSION : 20
OPERATOR : +
NUMBER/EXPRESSION : 5
Result = 25
```

Question 37

Online C Compiler

Run

lexer.l — Q37 - Frequency of Given Word

```
%{
#include <stdio.h>
#include <string.h>
int count=0;
char target[50];
}%
[a-zA-Z]+ {
    if(strcmp(yytext,target)==0) count++;
}
[ \t\n]+ ;
. ;
}%
int main(){
    printf("Enter word: ");
    scanf("%49s",target);
    getchar();
    printf("Enter sentence: ");
    yylex();
    printf("Frequency of %s = %d\n",target,count);
    return 0;
}
int yywrap(){ return 1; }
```

Output

```
Enter word:
lex
Enter sentence:
lex is simple and lex is useful
Frequency of lex = 2
```

Question 38

Online C Compiler

Run

lexer.l — Q38 - Find Length of Longest Word

```
%{
#include <stdio.h>
#include <string.h>
int max=0;
}%
%%
[a-zA-Z]+ {
    if(strlen(yytext)>max) max=strlen(yytext);
}
[ \t\n]+ ;
. ;
%%
int main(){ printf("Enter sentence: "); yylex();
printf("Length of longest word = %d\n",max); return 0; }
int yywrap(){ return 1; }
```

Output

```
Enter sentence:
LexicalAnalyzer programming language
Length of longest word = 16
```

Question 39

Online C Compiler

Run

lexer.l — Q39 - Replace Word in a File

```
%{
#include <stdio.h>
#include <string.h>
char oldword[]="cat";
char newword[]="dog";
}%
%%
[a-zA-Z]+ {
    if(strcmp(yytext,oldword)==0) printf("%s",newword);
    else printf("%s",yytext);
}
[ \t\n]+ { printf("%s",yytext); }
. { printf("%s",yytext); }
%%
int main(){ yylex(); return 0; }
int yywrap(){ return 1; }
```

Output

```
Input file:
The cat is a small cat.
Output file:
The dog is a small dog.
```

Question 40

Online C Compiler

Run

lexer.l — Q40 - FLEX Token Separation

```
%{
#include <stdio.h>
%}
%%
#include                { printf("PREPROCESSOR :
%s\n",yytext); }
"int"|"float"|"char"|"void"|"return"
                        { printf("KEYWORD      :
%s\n",yytext); }
[a-zA-Z_][a-zA-Z0-9_]* { printf("IDENTIFIER   :
%s\n",yytext); }
[0-9]+                  { printf("NUMBER       :
%s\n",yytext); }
\"[^\"]*"               { printf("STRING        :
%s\n",yytext); }
[+*/=-]                 { printf("OPERATOR     :
%s\n",yytext); }
[{}();,<,>]              { printf("SPECIAL      :
%s\n",yytext); }
[ \t\n]+ ;
. ;
%%
int main(){ yylex(); return 0; }
int yywrap(){ return 1; }
```

Output

```
Input Source Program: sample.c
#include<stdio.h>
void main()
{
    int a,b,c = 30;
    printf("hello");
}

PREPROCESSOR : #include
SPECIAL      : <
IDENTIFIER    : stdio
SPECIAL      : >
KEYWORD       : void
IDENTIFIER    : main
SPECIAL      : (
SPECIAL      : )
SPECIAL      : {
KEYWORD       : int
IDENTIFIER    : a
SPECIAL      : ,
IDENTIFIER    : b
SPECIAL      : ,
IDENTIFIER    : c
OPERATOR      : =
NUMBER        : 30
SPECIAL      : ;
IDENTIFIER    : printf
SPECIAL      : (
STRING        : "hello"
```